

CYBERSECURITY FOR PROFESSIONALS BOOTCAMP

HANDS-ON LAB

Threat Hunting and Detection Engineering

Week 6, Days 4–5 • Browser-Based Lab • Individual Lab

All answers have been **Bolded**

1. Before You Start

This lab continues directly from the forensic investigation you completed on Day 3. It runs entirely in your web browser. There is nothing to install, no virtual machine to connect to, no account to create, and no password to request. Everything you need is printed inside this document. The websites confirm what you already have; they do not hold anything you cannot see here.

1.1 What you need

Item	Detail
A web browser	Chrome, Edge, Firefox or Safari. Any current version is fine.
A spreadsheet	Excel, Google Sheets or LibreOffice Calc. Most of Day 4 happens here.
A plain text editor	Notepad, TextEdit, VS Code, or anything that saves .txt files. You will write detection rules on Day 5.
A document editor	Word, Google Docs or anything that produces a PDF for your reports.
Your Day 3 work	Your IOC list and forensic report. If you did not finish, Section 2 gives you everything you need.

You can do this lab even if Day 3 did not go well

Section 2 restates every finding from the Day 3 investigation: the timeline, the indicators, the root cause.

If you finished Day 3, use your own work and treat Section 2 as a check on it.

If you did not, use Section 2 as your starting point. You are not behind, and nothing in Days 4 and 5 is blocked.

1.2 The three websites you will use

Tool	Address	What you use it for
CyberChef	gchq.github.io/CyberChef	Pulling indicators out of large log exports, counting occurrences, and filtering. Published by GCHQ. Runs entirely inside your browser - nothing you type is ever uploaded anywhere.
MITRE ATT&CK	attack.mitre.org	The public catalogue of attacker techniques. You map every step of this intrusion to a technique identifier so other teams can understand it without reading your whole report.
Sigma specification	sigmahq.io	The open, vendor-neutral format for writing detection rules. You write four rules on Day 5. Reference only - you write the rules in a text editor.

If a website will not load

Appendix A.1 prints every ATT&CK technique used in this lab, with its identifier, official name and tactic. Appendix A.2 prints the full Sigma field reference and a complete worked rule.

You can finish this entire lab from the printed appendices. Note in your report which source you used.

You are never blocked. Move on and keep working.

1.3 Scope and ethics

Read this before you open anything

Everything in this lab is evidence that has already been collected for you. You are analysing it, not generating it.

Do not point any scanning tool at any host, website or address anywhere. Not the addresses in this document, not your employer's network, not a site you found online.

The two external addresses in this incident belong to the scenario. Do not visit them, scan them, look them up in a service that will connect to them, or paste them into any site that performs live lookups. Reputation services that actively probe a target are not appropriate here.

The internal addresses are private ranges that do not exist on the public internet.

Analysts get fired and prosecuted for touching things they were not authorised to touch. The habit of checking your authorisation first matters more than any tool in this lab.

2. Where We Left Off

On Day 3 you investigated MTB-WEB01 after the Meridian Trust Bank SOC raised alert SOC-2026-09-02-0416. This is what that investigation established.

2.1 The confirmed intrusion on MTB-WEB01

Time (UTC)	What happened	Technique
02 Sep 02:12:33	Path traversal request returns the contents of /etc/passwd	Exploitation of unpatched Apache 2.4.49
02 Sep 02:15:22	Webshell written to /var/www/html/uploads/style.php	Webshell dropped
02 Sep 02:17:40	Webshell used for the first time	Command execution as www-data
02 Sep 02:21:05	pkexec opens a root session for uid 33 (www-data)	Local privilege escalation
02 Sep 02:23:11	Account svc-backup created with UID 0	Backdoor account
02 Sep 02:24:48	Attacker SSH key added to /root/.ssh/authorized_keys	Persistent remote access
02 Sep 02:25:57	Beacon written to /usr/local/bin/apache-worker	Command and control tooling
02 Sep 02:26:30	Cron entry /etc/cron.d/apache-logrotate runs it every 5 minutes	Persistence
02 Sep 02:34:55	Customer data archived into /tmp/.ICE-unix and later deleted	Collection and staging
02 Sep 02:41:19	Shell history cleared on one account but not the other	Covering tracks

Time (UTC)	What happened	Technique
02 Sep 09:47:12	SOC alerts on the repeated outbound connection	Detection

2.2 The confirmed root cause

Element	Detail	CVE	Day 1-2 finding
Initial access	Apache 2.4.49 path traversal leading to remote code execution	CVE-2021-41773 <i>CVE-2021-42013</i>	MTB-0001 <i>MTB-0002</i>
Enabling misconfiguration	ScriptAlias to /cgi-bin/ combined with Require all granted on the filesystem root	-	Evidence pack
Privilege escalation	polkit pkexec (PwnKit) against policykit-1 0.105-31	CVE-2021-4034	MTB-0003

2.3 The Day 3 indicator list

These are the indicators your forensic report handed to the wider security team. They are your starting point today.

Type	Indicator	Where it was found
Network	185.220.101.47	Beacon destination, exploit source, SSH source
Network	TCP port 8443 outbound	Beacon destination port
File path	/var/www/html/uploads/style.php	Webshell
File path	/usr/local/bin/apache-worker	Beacon
File path	/etc/cron.d/apache-logrotate	Persistence
File path	/tmp/.ICE-unix	Exfiltration staging directory
Account	svc-backup (UID 0)	Backdoor account
SSH key	Key comment attacker@blackfin	/root/.ssh/authorized_keys
User agent	curl/7.81.0	Exploitation requests

The two file hashes recorded at acquisition are reproduced below on their own lines so they can be copied cleanly.

```
/var/www/html/uploads/style.php
2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c

/usr/local/bin/apache-worker
1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b
```

What that report did not tell you

A forensic report on one host answers what happened to that host. It does not answer the two questions the bank actually needs answered next.

How long has this been going on? Your timeline started at 02:12 on 2 September because that is when the disk evidence started. That is not the same thing as when the intrusion started.

Where else is the attacker? You examined one server. Meridian Trust runs more than one.

Answering those two questions is Day 4.

3. How to Work Through This Lab

3.1 Time plan

These are guide times, not limits. If a section is taking dramatically longer than the guide, check Appendix A.4 before assuming you are stuck.

Session	Section	What you are doing	Guide time
Day 4	4	Read the hunt brief and the fleet inventory	30 minutes
Day 4	5	Extend the timeline using network telemetry	1 to 2 hours
Day 4	6	Hunt the indicators across the fleet	2 to 3 hours
Day 4	7	Rewrite your indicators using the Pyramid of Pain	1 hour
Day 4	8	Write the incident scoping report	1 to 2 hours
Day 5	9	Map the intrusion to MITRE ATT&CK	1 to 2 hours
Day 5	10	Write four Sigma detection rules	2 to 3 hours
Day 5	11	Containment, eradication and recovery plan	1 to 2 hours
Day 5	12	Post-incident review	1 hour

3.2 Set up your workbook before you start

Where	Name it	What goes in it
Spreadsheet tab 1	Egress Log	The perimeter firewall export from Section 5.1
Spreadsheet tab 2	Fleet Hunt	One row per host, one column per indicator, filled in as you hunt
Spreadsheet tab 3	Extended Timeline	The merged timeline covering the full intrusion, not just 2 September
Spreadsheet tab 4	IOC Tiers	Your rewritten indicator list, sorted by Pyramid of Pain tier
Spreadsheet tab 5	ATT&CK Map	Technique identifier, name, tactic, and your supporting evidence
Text file 1 to 4	mtb_detect_01.yml to mtb_detect_04.yml	Your four Sigma rules, one per file
Document 1	MTB Incident Scoping Report	Your Day 4 deliverable
Document 2	MTB Post-Incident Review - Case MTB-IR-2026-0902	Your Day 5 deliverable

3.3 What a good answer looks like

As on Days 1 to 3, most of the marks are for reasoning. Here is the same hunting conclusion written three ways.

Standard	Example write-up
Not acceptable	MTB-FILE01 is also compromised.
Passing	MTB-FILE01 is also compromised. Its auth log shows an SSH login from MTB-WEB01 at 02:52 on 2 September using the svc-backup account, and it is running a beacon process.
Strong	MTB-FILE01 is compromised with high confidence. Its auth log records an accepted publickey login for svc-backup from 10.50.2.24 (MTB-WEB01) at 02:52:14 on 2 September, twenty-eight minutes after the attacker gained root on MTB-WEB01 and harvested the SSH private key. The process telemetry shows a persistent outbound session to 185.220.102.244:443 attributed to /usr/local/sbin/syslog-forward, held open by a systemd unit rather than a cron entry. Note the tradecraft differs from MTB-WEB01: a different C2 address and a different persistence mechanism. An indicator-only hunt for 185.220.101.47 and /etc/cron.d/apache-logrotate would have missed this host entirely, which is the argument for hunting on behaviour rather than on addresses.

The pattern to copy

State the conclusion and your confidence. Cite the specific log line with its timestamp and source host. Connect it to what you already knew. Note what is different or surprising. Say what that difference means for how the hunt should be run.

Every hunting conclusion in your scoping report should follow that shape.

4. Day 4, Part 1: The Hunt Brief

Your forensic report went to the Head of Security Operations at 08:00 this morning. This came back.

From : Head of Security Operations, Meridian Trust Bank
 To : Incident Response
 Subject : MTB-IR-2026-0902 - scoping required before we brief the CISO

Good report. Two things before this goes any further.

1. I do not accept that this started at 02:12 on 2 September. That is when your disk evidence starts, which is not the same thing. The Apache access log on that image only went back to 1 September because of log rotation. Get me the real start date from the perimeter telemetry, which we retain for 90 days.
2. You examined one server. We run five in that segment. The attacker had root on a host in our DMZ for at least seven hours and there was an SSH private key sitting on it. Assume they used it. Tell me how many hosts are affected, and tell me what you checked on the ones you are calling clean.

I need a scoping report by end of day. Not a full incident report.
 Scope: how far back, how many hosts, what data.

4.1 What threat hunting actually is

Up to now you have been reactive: an alert fired, and you investigated it. Hunting is the opposite. You start from a hypothesis about what an attacker would do, and you go looking for evidence of it in places nobody has alerted on.

A hunt hypothesis is a specific, testable statement. "Check if we are compromised" is not a hypothesis. "If the attacker harvested the SSH key from MTB-WEB01, other hosts in the server segment will show accepted publickey logins originating from 10.50.2.24 around or after 02:24 on 2 September" is a hypothesis, because it tells you exactly which log, which field and which time window to look at.

Your hypotheses for today	What you will test it against
The intrusion began before 2 September	Perimeter firewall egress log, Section 5.1
The attacker moved from MTB-WEB01 to other hosts using the harvested SSH key	Fleet authentication logs, Section 6.2
Other hosts are running a beacon	Fleet process and network telemetry, Section 6.3
The attacker used the same tooling everywhere	Everything above, and this one is wrong

A hunt that finds nothing is still a result

Two of the five hosts in this environment are genuinely clean. Proving that is as valuable as finding a compromise, and your scoping report must say what you checked on them.

"We found no evidence on MTB-DC01" is worthless on its own. "We reviewed MTB-DC01 authentication logs for the full window, checked for all six file-path indicators and reviewed its egress records, and found no evidence of compromise" is a finding a CISO can act on.

The difference is whether the reader can tell what you would have caught.

4.2 The fleet

These are the five hosts in the server segment, plus the two workstation addresses that appear in the logs. This is your hunting scope.

Host	Address	Role	Status at start of hunt
MTB-WEB01	10.50.2.24	DMZ customer portal	CONFIRMED COMPROMISED (Day 3)
MTB-APP01	10.50.2.25	Application and file transfer	Unknown
MTB-FILE01	10.50.2.26	Internal file server	Unknown
MTB-DC01	10.50.2.10	Windows domain controller	Unknown
MTB-DVWA01	10.50.2.30	Test web application	Unknown
Workstation	10.50.1.31	Analyst workstation, mtbadmin	Seen in Day 3 logs, legitimate
Workstation	10.50.1.44	Analyst workstation	Seen in Day 3 logs, legitimate

5. Day 4, Part 2: Extending the Timeline

Your Day 3 timeline started on 2 September because that is where the disk evidence started. The Apache access log on the image began at 14:02 on 1 September, and the file system timeline could only show you files that still existed when the image was taken.

Neither of those tells you when the attacker first arrived. For that you need telemetry that lives somewhere the attacker could not reach and that is retained longer than the host's own logs.

5.1 The perimeter firewall egress log

Meridian Trust retains 90 days of perimeter connection records. This is the extract for the server segment, filtered to external destinations, covering 26 August to 2 September. Sessions to normal business destinations have been removed.

date	time	src_ip	src_host	dst_ip	dport	proto	action	bytes_out	bytes_in	sessions
2026-08-26	03:41:02	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1204	880	1
2026-08-28	01:52:17	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	1
2026-08-28	01:52:19	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	214
2026-08-28	01:58:44	185.220.101.47	-	10.50.2.24	443	tcp	deny	0	0	6
2026-08-28	02:03:11	185.220.101.47	-	10.50.2.24	22	tcp	deny	0	0	3
2026-08-30	02:07:55	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	12
2026-08-30	02:09:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	842	119	1
2026-08-30	02:14:40	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	4
2026-09-01	09:15:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1198	874	1
2026-09-01	09:20:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1201	881	1
2026-09-01	09:25:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1197	879	1
2026-09-02	02:08:14	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	9
2026-09-02	02:12:33	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	6
2026-09-02	02:24:48	185.220.101.47	-	10.50.2.24	22	tcp	allow	0	0	1
2026-09-02	02:31:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	611	88	1
2026-09-02	02:36:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	604	88	1
2026-09-02	02:38:02	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	2884416	412	1
2026-09-02	02:41:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	598	88	1
2026-09-02	02:57:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	744	102	1
2026-09-02	03:27:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	739	102	1
2026-09-02	03:57:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	741	102	1
2026-09-02	04:27:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	740	102	1
2026-09-02	05:12:19	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	9917722	508	1
2026-09-02	05:27:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	742	102	1
2026-09-02	09:20:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1203	877	1
2026-09-02	09:25:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1199	883	1

5.2 Reading an egress log

Column	What it tells you
src_ip and src_host	Where the connection came from. A blank src_host means the source is external and not one of ours.
dst_ip and dport	Where it was going. For outbound rows this is the interesting half.
action	Whether the firewall permitted it. A deny row still tells you someone tried.
bytes_out	How much data left the bank. This column is where exfiltration becomes visible.
sessions	How many separate connections this row summarises. A single row with a high session count is a scan, not a conversation.

Three things in this log are worth stopping on

One row has a session count in the hundreds against a single host and port, with zero bytes transferred in either direction. Zero bytes means no conversation actually happened. What activity produces hundreds of connections that carry no data?

Two rows have a bytes_out value thousands of times larger than every other row. Note both, and note which host each one comes from.

One host produces a perfectly regular outbound pattern on a five-minute interval for the entire period, including days before the intrusion. Regularity is a beacon characteristic, so look at it - and then work out why it is not one.

5.3 Step by step

1. Copy the egress log into your **Egress Log** spreadsheet tab. It is space-separated; use your spreadsheet's text-to-columns feature to split it.

Revised start of intrusion: August 28, 2026 at 01:52:17 UTC.

2. Sort by date and time. Identify the earliest row involving 185.220.101.47 and record that timestamp. This is your revised start of intrusion.

The perimeter telemetry extended the known intrusion timeline approximately 5 days earlier than the Day 3 disk evidence.

3. Compare that date against the start of your Day 3 timeline. Calculate the difference in days. This is how much of the intrusion your disk evidence could not see.

The disk image could not show the full intrusion because the Apache access logs had rotated and only went back to September 1. In addition, the perimeter firewall telemetry was stored outside MTB-WEB01, so that evidence was not contained on the forensic disk image and could not be removed or altered by activity on the compromised server.

4. Explain in writing why the disk image could not show you this activity. Two reasons apply: one about log rotation, one about where the evidence lives.

MTB-WEB01 established an outbound connection to the attacker infrastructure on August 30 at 02:09:12 UTC. This proves that the host had already been compromised before the September 2 exploitation activity observed during Day 3.

5. Find the row on 30 August where MTB-WEB01 first connects outbound to the attacker. Note what this means: the compromise on 2 September was not the first one.

External destination	Internal host	Meaning
185.220.101.47	MTB-WEB01	Attacker/C2 infrastructure
185.220.102.244	MTB-FILE01	Second suspicious C2 destination
52.168.117.171	MTB-DVWA01	Regular external communication; later confirmed as approved monitoring

6. Identify every distinct external destination address in the log. There are three. For each, note which internal hosts talked to it.

Timestamp	Host	Bytes Out	Approx. MB
2026-09-02 05:12:19	MTB-FILE01	9,917,722	9.92 MB
2026-09-02 02:38:02	MTB-WEB01	2,884,416	2.88 MB

The largest transfer was approximately 9.92 MB from MTB-FILE01 at 05:12:19 UTC. The second largest was approximately 2.88 MB from MTB-WEB01 at 02:38:02 UTC. These transfers are significant because they may represent data exfiltration.

7. Sort by bytes_out, largest first. Record the two largest rows with their host, timestamp and volume. Convert the byte counts into megabytes.
8. One of those two large transfers is from a host you have not investigated yet. Note it. You will confirm it in Section 6.

MTB-FILE01 is the previously uninvestigated host associated with the largest outbound transfer and requires further investigation in Section 6.

5.4 Record: the extended timeline

Date and time (UTC)	Host	Event	Evidence source
2026-08-28 01:52:17	MTB-WEB01	First observed attacker connection to MTB-WEB01 over TCP port 80	Perimeter firewall log
2026-08-30 02:09:12	MTB-WEB01	First outbound connection to attacker infrastructure on TCP 8443, showing compromise already existed	Perimeter firewall log
2026-09-02 02:12:33	MTB-WEB01	Path traversal successfully returns /etc/passwd	Apache / Day 3 evidence
2026-09-02 02:15:22	MTB-WEB01	Webshell written to /var/www/html/uploads/style.php	Day 3 forensic evidence
2026-09-02 02:21:05	MTB-WEB01	pkexec opens root session for www-data	Day 3 forensic evidence
2026-09-02 02:23:11	MTB-WEB01	svc-backup account created with UID 0	Day 3 forensic evidence
2026-09-02 02:24:48	MTB-WEB01	Attacker SSH key added for persistent access	Day 3 forensic evidence
2026-09-02 02:38:02	MTB-WEB01	Approximately 2.88 MB transferred outbound	Perimeter firewall log
2026-09-02 05:12:19	MTB-FILE01	Approximately 9.92 MB transferred to suspicious external destination	Perimeter firewall log
2026-09-02 09:47:12	MTB-WEB01	SOC alert reaches a human analyst	SOC / Day 3 evidence

Merge this with your Day 3 timeline rather than keeping them separate. The report needs one timeline covering the whole intrusion.

6. Day 4, Part 3: Hunting the Fleet

You now know the intrusion is older than you thought and that at least one other host has been talking to attacker infrastructure. This section is where you confirm which hosts are affected and, just as importantly, which are not.

6.1 How to run a hunt across many hosts

Build a grid. One row per host, one column per indicator. Fill in every cell, including the ones where the answer is no. The empty cells at the end are what tells you where you still have no visibility.

Host	185.220.101.47	185.220.102.244	apache-worker	cron entry	svc-backup	Verdict
MTB-WEB01	YES	NO	YES	YES	YES	COMPROMISED
MTB-APP01	NO	NO	NO	NO	NO	No evidence
MTB-FILE01	NO	YES	NO	NO	YES	COMPROMISED
MTB-DC01	NO	NO	NO	NO	NO	CLEAN
MTB-DVWA01	NO	NO	NO	NO	NO	CLEAN

The trap in this grid

If you fill in only these columns you will reach the wrong answer on one host, because the attacker did not use the same tooling everywhere.

When you finish the grid, add columns for whatever new indicators you discover along the way. A hunt grid that never grows is a hunt that only found what it already knew about.

6.2 Fleet authentication logs

Authentication records for the four uninvestigated hosts, covering 2 September. Routine internal activity has been left in so you can see what normal looks like.

MTB-APP01 (10.50.2.25)

```
Sep  2 07:02:11 mtb-app01 sshd[2201]: Accepted publickey for mtbadmin from 10.50.1.31 port 51002
Sep  2 07:44:56 mtb-app01 sshd[2240]: Accepted publickey for mtbdeploy from 10.50.1.44 port 39114
Sep  2 11:20:03 mtb-app01 sshd[2388]: Failed password for invalid user oracle from 10.50.1.99 port 44120
Sep  2 11:20:07 mtb-app01 sshd[2388]: Failed password for invalid user oracle from 10.50.1.99 port 44120
Sep  2 11:20:12 mtb-app01 sshd[2388]: Connection closed by invalid user oracle 10.50.1.99
```

MTB-FILE01 (10.50.2.26)

```
Sep  2 06:25:01 mtb-file01 CRON[1801]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 02:52:14 mtb-file01 sshd[1655]: Accepted publickey for svc-backup from 10.50.2.24 port 52118
```

```

Sep  2 02:52:14 mtb-file01 sshd[1655]: pam_unix(sshd:session): session opened for user svc-backup(uid=0)
Sep  2 02:54:39 mtb-file01 useradd[1668]: new user: name=svc-backup, UID=0, GID=0, home=/home/svc-backup
Sep  2 02:54:39 mtb-file01 useradd[1668]: WARNING: UID 0 is already in use by user root
Sep  2 02:56:02 mtb-file01 systemd[1]: Reloading.
Sep  2 02:56:03 mtb-file01 systemd[1]: Started System Log Forwarder.
Sep  2 08:11:47 mtb-file01 sshd[1902]: Accepted publickey for mtbadmin from 10.50.1.31 port 49882

```

MTB-DC01 (10.50.2.10)

```

09/02/2026 07:14:22 Event 4624 Logon Type 3 Account: MTB\\mtbadmin Source: 10.50.1.31
09/02/2026 07:51:09 Event 4624 Logon Type 2 Account: MTB\\Administrator Source: 10.50.2.10
09/02/2026 09:02:44 Event 4624 Logon Type 3 Account: MTB\\svc-sql Source: 10.50.2.25
09/02/2026 13:30:17 Event 4625 Logon Type 3 Account: MTB\\jsmith Source: 10.50.1.44
09/02/2026 13:30:41 Event 4624 Logon Type 3 Account: MTB\\jsmith Source: 10.50.1.44

```

MTB-DVWA01 (10.50.2.30)

```

Sep  2 08:40:15 mtb-dvwa01 sshd[3301]: Accepted publickey for mtbdeploy from 10.50.1.44 port 40221
Sep  2 15:02:38 mtb-dvwa01 sshd[3455]: Accepted publickey for mtbdeploy from 10.50.1.44 port 40990

```

6.3 Fleet process and network telemetry

Endpoint telemetry showing long-running processes holding external network sessions, captured at 10:20 on 2 September across all five hosts.

host	pid	user	process	parent	remote_endpoint	state	uptime
MTB-WEB01	1244	root	/bin/bash /usr/local/bin/apache-worker	cron	185.220.101.47:8443	ESTAB	07:49
MTB-WEB01	901	root	/usr/sbin/apache2 -k start	systemd	-	-	8d
MTB-APP01	2105	root	/usr/sbin/vsftpd	systemd	-	-	8d
MTB-APP01	2110	root	/usr/sbin/sshd -D	systemd	-	-	8d
MTB-FILE01	1671	root	/usr/local/sbin/syslog-forward	systemd	185.220.102.244:443	ESTAB	07:24
MTB-FILE01	1520	root	/usr/sbin/smbd -D	systemd	-	-	8d
MTB-DC01	4188	SYSTEM	C:\\Windows\\System32\\svchost.exe	services	-	-	8d
MTB-DC01	4402	SYSTEM	C:\\Windows\\System32\\lsass.exe	wininit	-	-	8d
MTB-DVWA01	3102	monagent	/opt/monagent/bin/monagent --daemon	systemd	52.168.117.171:443	ESTAB	31d
MTB-DVWA01	3010	root	/usr/sbin/apache2 -k start	systemd	-	-	8d

6.4 The asset register entry for MTB-DVWA01

Before you make a judgement on the host with the regular five-minute outbound pattern, read its asset record.

```

Asset:          MTB-DVWA01
Owner:          Application Security - test environment
Environment:    Non-production, isolated test
Installed agents: monagent 4.2.1 (Meridian Trust estate monitoring)
                  Check-in interval: 300 seconds
                  Check-in endpoint: 52.168.117.171:443 (approved egress allowlist)
Deployed:       2026-08-02

```

6.5 Step by step

1. Fill in the grid in Section 6.1 for all five hosts, using the logs and telemetry above. Every cell gets an answer.

(Completed in 6.1)

2. Identify the host that shows an SSH login originating from MTB-WEB01. Record the exact timestamp, the account used, and how long after the MTB-WEB01 root compromise it occurred.

MTB-FILE01 received an accepted public-key SSH login from MTB-WEB01 (10.50.2.24) at 02:52:14 UTC on September 2 using the svc-backup account. MTB-WEB01 gained root access at 02:21:05 UTC, so the lateral movement occurred 31 minutes and 9 seconds after root compromise.

3. Explain how the attacker was able to authenticate to that second host. Your Day 3 evidence tells you what was sitting on MTB-WEB01 that made this possible.

The attacker was able to authenticate to MTB-FILE01 because an SSH private key was available on the compromised MTB-WEB01 system. After obtaining root access, the attacker could use that trusted key to authenticate to another server without needing the user's password.

4. On that second host, identify the beacon. Note three ways it differs from the one on MTB-WEB01: the destination address, the file path, and the persistence mechanism.

MTB-FILE01 used different tradecraft from MTB-WEB01. Its beacon connected to 185.220.102.244:443 instead of 185.220.101.47:8443, used /usr/local/sbin/syslog-forward instead of /usr/local/bin/apache-worker, and persisted through a systemd service instead of cron. This demonstrates why hunting only for exact IP addresses and filenames can miss additional compromised systems.

5. Add columns to your grid for the new indicators you just found, and re-check every host against them.

Host	syslog-forward	System Log Forwarder	SSH from MTB-WEB01
MTB-WEB01	NO	NO	N/A
MTB-APP01	NO	NO	NO
MTB-FILE01	YES	YES	YES
MTB-DC01	NO	NO	NO
MTB-DVWA01	NO	NO	NO

6. Look at MTB-DVWA01. It has a persistent external session on a regular five-minute interval, which is exactly the pattern that got MTB-WEB01 flagged. Decide whether it is compromised, and write down the specific evidence that settles it.

MTB-DVWA01 is assessed as clean. Although it maintains a regular five-minute external connection, the asset register confirms that /opt/monagent/bin/monagent is an approved Meridian Trust monitoring agent. Its documented check-in interval is 300 seconds, and its destination 52.168.117.171:443 is on the approved egress allowlist. Therefore, the periodic activity is legitimate rather than malicious beaconing.

7. For the two hosts you conclude are clean, write down exactly what you checked. Not "nothing found" - list the indicators and the log sources.

MTB-DC01 authentication logs, process/network telemetry, known C2 indicators, svc-backup activity, and evidence of connections originating from MTB-WEB01 were reviewed. No suspicious authentication, malicious external connection, or known attack indicator was identified.

MTB-DVWA01 authentication logs, process/network telemetry, known C2 indicators, and the asset register were reviewed. No svc-backup login or connection to attacker infrastructure was identified. Its only persistent external session was confirmed as the approved monagent monitoring service.

8. Look at the failed logins on MTB-APP01 at 11:20. Decide whether they are related to this incident and justify your answer using the source address and the timing.

The failed oracle logins on MTB-APP01 at 11:20 UTC do not appear to be related to this incident. They originated from 10.50.1.99, not MTB-WEB01, used an unrelated invalid account, and occurred after the known lateral movement and after the SOC alert. However, because 10.50.1.99 is not one of the known legitimate workstations listed in the lab, the activity should be investigated separately rather than assumed to be harmless.

The point of the MTB-DVWA01 question

Periodicity is a beacon characteristic. It is not proof of one. Plenty of legitimate software checks in on a fixed schedule, and every monitoring agent in your estate looks like a beacon if you squint.

What separates them is not the pattern. It is whether the process, the destination and the schedule are all accounted for by something you approved.

An analyst who cannot tell the difference generates false alarms until people stop reading their alerts. That is a worse outcome than missing one beacon.

7. Day 4, Part 4: Rewriting Your Indicators

Your hunt just demonstrated something uncomfortable. The indicator list you produced on Day 3 would have missed MTB-FILE01 completely, because the attacker used a different address, a different filename and a different persistence mechanism on that host.

That is not a failure of your investigation. It is a property of indicators, and there is a well-known model for thinking about it.

7.1 The Pyramid of Pain

Introduced by David Bianco in 2013, the Pyramid of Pain ranks indicator types by how much difficulty you cause an attacker when you detect and block them. The higher up you can detect, the harder it is for the attacker to carry on.

Tier	Indicator type	Pain caused	Example from this incident
6 (top)	TTPs	Tough	A beacon process launched by a persistence mechanism, holding a long-lived outbound session on a fixed interval
5	Tools	Challenging	The specific beacon implementation, whichever filename it is given
4	Network or host artifacts	Annoying	A root-owned process in /usr/local/ with a persistence entry created outside a change window
3	Domain names	Simple	(none observed in this incident)

Tier	Indicator type	Pain caused	Example from this incident
2	IP addresses	Easy	185.220.101.47 and 185.220.102.244
1 (bottom)	Hash values	Trivial	The SHA-256 of style.php and apache-worker

Why this incident proves the model

You blocked 185.220.101.47 after Day 3. The attacker was already using 185.220.102.244 on the second host, at no cost to them - changing an address takes seconds.

You hashed apache-worker. On MTB-FILE01 the same capability is called syslog-forward, which produces a different hash.

What did not change between the two hosts: a root-owned binary outside the package manager's control, started by a persistence mechanism, holding a long-lived outbound session at a fixed interval. Detect that and the attacker has to redesign their tooling, not edit a config file.

This is why Day 5 asks you to write detections for behaviour rather than for addresses.

7.2 Step by step

1. List every indicator from Day 3 and every new one from today. There should be at least sixteen.
2. Assign each one a Pyramid tier from the table in Section 7.1.
3. For each indicator at tier 1 or 2, write down how long it would take the attacker to change it and carry on. Be honest - the answer is usually seconds or minutes.
4. Write three tier 4 to 6 descriptions of this intrusion that do not name a single address, filename or hash. These become the basis of your Day 5 detection rules, so make them specific enough to test.

1. A root-owned executable outside normal package-managed directories is launched through a persistence mechanism and maintains a long-lived outbound network session.

2. A privileged service account authenticates by public-key SSH from one server-segment host to another shortly after the source host is compromised.

3. A script is written into a web-accessible upload directory after exploitation of a public-facing web service and is then used to execute commands on the server.

5. Explain in two or three sentences why an indicator list is still worth producing even though most of it sits at the bottom two tiers.

Indicator lists are still useful because IP addresses and hashes are quick to search for, block, and share with other security teams. However, attackers can change these indicators easily, so they should be combined with higher-level behavioral detections that are harder to evade.

7.3 Record: tiered indicator list

Tier	Indicator or behaviour description	Source	Attacker cost to change
1	Style.php= 2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68f f7a025e975c20c	Day 3 forensic evidence	Seconds: modify the file to produce a new hash
1	apache-worker= 1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef0601 4b1a00ea4d15d4b	Day 3 forensic evidence	Seconds: modify/recompile the file
2	185.220.101.47	Day 3 / firewall log	Seconds–minutes: change C2 IP
2	185.220.102.244	Day 4 firewall telemetry	Seconds–minutes: change C2 IP
4	TCP port 8443 outbound	Day 3 network evidence	Minutes: use another port
4	/var/www/html/uploads/style.php	Day 3 forensic evidence	Minutes: rename or relocate webshell
4	/usr/local/bin/apache-worker	Day 3 process evidence	Minutes: rename or relocate beacon
4	/etc/cron.d/apache-logrotate	Day 3 forensic evidence	Minutes: change cron entry or persistence method
4	/tmp/.ICE-unix	Day 3 forensic evidence	Minutes: use another staging directory
4	svc-backup account with UID 0	Day 3 / auth logs	Minutes: create a different account
4	SSH key comment attacker@blackfin	Day 3 authorized_keys evidence	Minutes: replace or alter the key
4	User agent curl/7.81.0	Day 3 Apache logs	Seconds: change the user agent
4	/usr/local/sbin/syslog-forward	MTB-FILE01 process telemetry	Minutes: rename or relocate binary
4	System Log Forwarder systemd service	MTB-FILE01 system logs	Minutes–hours: change persistence mechanism
4	TCP port 443 outbound used by FILE01 beacon	Day 4 network telemetry	Minutes: change connection port

Tier	Indicator or behaviour description	Source	Attacker cost to change
6	Public-key SSH lateral movement from a compromised server using a privileged service account	MTB-FILE01 authentication logs	Significant: change lateral-movement technique

8. Day 4, Part 5: The Incident Scoping Report

The Head of Security Operations asked three questions: how far back, how many hosts, what data. Answer those three and nothing else. This is not a second forensic report.

8.1 What goes in it

1. State the revised start of intrusion, with the evidence, and state how many days earlier it is than your Day 3 timeline suggested. Explain in one sentence why the disk evidence could not show this.
2. State the number of compromised hosts and name them, each with the evidence that confirms it and your confidence level.
3. State which hosts you assess as clean, and for each one list what you checked. A reader must be able to tell what your check would and would not have caught.
4. State the data at risk. Two large outbound transfers appear in the egress log. Give the host, timestamp and volume of each, and say what you believe left the bank based on what each host stores.
5. State the lateral movement path, from initial access through to the second host, naming the mechanism at each step.
6. State what you still do not know, and what evidence you would need to close each gap. A scoping report that claims certainty it does not have is worse than one that names its blind spots.
7. Write a short executive summary, three to five sentences, that answers the three questions in plain language.

Dwell time is the number that will be quoted

Dwell time is the gap between the attacker's first activity and the moment you detected them. It is the single figure that gets repeated to the board, the regulator and the auditor, because it measures the security programme rather than the attacker.

Calculate it from your revised start date, not from the disk timeline. Show your working in the report, because the first thing anyone senior will ask is how you arrived at that date.

8.2 Report structure to follow

Section	Contents
Executive summary	The investigation shows attacker activity began on August 28, 2026 at 01:52:17 UTC, approximately five days earlier than the Day 3 host evidence suggested. Two systems, MTB-WEB01 and MTB-FILE01, are confirmed compromised with high confidence. Approximately 2.88 MB of data left MTB-WEB01 and 9.92 MB left

Section	Contents
	MTB-FILE01 during suspicious outbound sessions. MTB-DC01 and MTB-DVWA01 are assessed as clean based on authentication, network, process, and indicator checks, while unrelated failed login activity on MTB-APP01 requires separate follow-up.
Revised timeline	The revised start of the intrusion is August 28, 2026 at 01:52:17 UTC, when perimeter firewall telemetry first recorded activity from the attacker infrastructure against MTB-WEB01 on TCP port 80. At 01:52:19, the same source generated 214 zero-byte sessions against the web server, consistent with vulnerability scanning. The Day 3 investigation originally began at September 2, 2026 at 02:12:33 UTC because that was the earliest relevant evidence remaining on the forensic image. The earlier activity was not visible because the Apache logs had rotated and because perimeter firewall telemetry was stored outside the compromised host. Dwell time: The first observed attacker activity occurred on August 28, 2026 at 01:52:17 UTC, and the SOC alert reached a human analyst on September 2, 2026 at 09:47:12 UTC. This gives a total dwell time of 5 days, 7 hours, 54 minutes, and 55 seconds.
Confirmed compromised hosts	MTB-WEB01 — High Confidence MTB-WEB01 is confirmed compromised. Evidence includes exploitation of the public-facing Apache server, creation and use of a webshell, privilege escalation to root, creation of the svc-backup UID 0 account, installation of an SSH key, cron-based persistence, and a root-owned beacon maintaining outbound communication. The host also transferred approximately 2.88 MB outbound at 02:38:02 UTC on September 2, shortly after customer data was archived for staging. MTB-FILE01 — High Confidence MTB-FILE01 is also confirmed compromised. At 02:52:14 UTC on September 2, it accepted a public-key SSH login for svc-backup originating from MTB-WEB01. At 02:54:39, a UID 0 svc-backup account was created, followed by creation of the System Log Forwarder systemd service. Process telemetry identified /usr/local/sbin/syslog-forward running as root and maintaining an established outbound connection to a second suspicious external destination. MTB-FILE01 later transferred approximately 9.92 MB outbound at 05:12:19 UTC. The tradecraft differed from MTB-WEB01: a different C2 address, different beacon path, and systemd rather than cron persistence. This confirms that exact IOC matching alone would have missed the second compromised system.

Section	Contents
Hosts assessed as clean	MTB-DC01 MTB-DC01 authentication logs, Windows process/network telemetry, known C2 indicators, svc-backup activity, and evidence of authentication originating from MTB-WEB01 were reviewed. The supplied evidence contains no suspicious authentication, malicious outbound session, or known indicator associated with this intrusion. Assessment: Clean based on available evidence. MTB-DVWA01 MTB-DVWA01 authentication logs, network/process telemetry, known attack indicators, and the asset register were reviewed. Although the host communicates externally on a regular five-minute interval, the connection is generated by the approved monagent monitoring service. The asset record confirms the documented 300-second check-in interval, approved monitoring destination, and legitimate process. Assessment: Clean based on available evidence. MTB-APP01 — No Evidence of This Incident / Further Review Required No evidence currently links MTB-APP01 to this intrusion. However, failed logins for the invalid oracle account from 10.50.1.99 at approximately 11:20 UTC should be investigated separately because that source is not one of the legitimate workstations documented in the supplied scope.
Lateral movement	The attacker initially targeted and compromised MTB-WEB01, exploiting the public-facing Apache service. After obtaining command execution, the attacker used pkexec to gain root privileges. With root-level access, the attacker had access to SSH credentials and trust relationships on MTB-WEB01. At 02:52:14 UTC, MTB-FILE01 accepted a public-key SSH login for svc-backup from MTB-WEB01. The lateral movement path was therefore: Public-facing web exploitation → webshell → root privilege escalation → SSH credential/key access → public-key SSH from MTB-WEB01 → MTB-FILE01 → UID 0 backdoor account → systemd persistence and beaconing.

Section	Contents												
Data at risk	Two unusually large outbound transfers were identified.												
	<table><tr><th>Host</th><th>Timestamp UTC</th><th>Volume</th><th>Assessment</th></tr><tr><td>MTB-WEB01</td><td>2026-09-02 02:38:02</td><td>2.88 MB</td><td>Likely customer data</td></tr><tr><td>MTB-FILE01</td><td>2026-09-02 05:12:19</td><td>9.92 MB</td><td>Likely internal file-server data unknown</td></tr></table>	Host	Timestamp UTC	Volume	Assessment	MTB-WEB01	2026-09-02 02:38:02	2.88 MB	Likely customer data	MTB-FILE01	2026-09-02 05:12:19	9.92 MB	Likely internal file-server data unknown
	Host	Timestamp UTC	Volume	Assessment									
	MTB-WEB01	2026-09-02 02:38:02	2.88 MB	Likely customer data									
	MTB-FILE01	2026-09-02 05:12:19	9.92 MB	Likely internal file-server data unknown									
	The MTB-WEB01 transfer occurred shortly after customer data was archived into a staging location, making customer information the likely contents of that transfer.												
MTB-FILE01 is an internal file server, so its much larger transfer may contain internal business or shared-file data. The supplied evidence does not identify the exact files transferred, so the specific contents and sensitivity remain an intelligence gap.													
Intelligence gaps	Exact contents of the MTB-FILE01 transfer: The network log shows approximately 9.92 MB left the host, but not which files were sent. File-access logs, audit records, staging artifacts, or more detailed network telemetry would be needed to determine the contents.												
	Exact contents of the MTB-WEB01 transfer: Customer data was staged shortly before the 2.88 MB transfer, creating a strong link, but packet capture or recovered archive contents would be required to prove exactly what left the environment.												
	Full SSH-key blast radius: The investigation confirms that the attacker moved from MTB-WEB01 to MTB-FILE01 using trusted SSH access, but the available evidence does not establish every system that trusted the exposed key. Reviewing authorized_keys, SSH configuration, and authentication logs across the environment would close this gap.												
	MTB-APP01 failed logins: The activity from 10.50.1.99 does not match the known intrusion, but its source and purpose remain unexplained. Additional endpoint and authentication investigation is required before determining whether it represents a separate security event.												

9. Day 5, Part 1: Mapping the Intrusion to MITRE ATT&CK

You have described this intrusion twice now: once as a forensic timeline and once as a scoping report. Both are written in Meridian Trust's own words. That is a problem the moment you need to talk to anyone outside your team.

MITRE ATT&CK is the shared vocabulary that solves it. It is a free, public catalogue of the things attackers actually do, each with a stable identifier. When you say T1505.003, every analyst, every vendor and every threat intelligence report means the same thing by it.

9.1 How ATT&CK is organised

Level	What it is	Example
Tactic	The attacker's goal at that moment. Why they are doing it. Fifteen of them, numbered TA####.	Persistence (TA0003)
Technique	How they achieve that goal. Numbered T####.	Server Software Component (T1505)
Sub-technique	A specific variant of a technique. Numbered T####.###.	Web Shell (T1505.003)

One thing that will confuse you if nobody warns you

ATT&CK renamed a tactic. What older training material, older tooling and most blog posts still call Defense Evasion is now called Stealth (TA0005).

There is also a newer tactic called Defense Impairment (TA0112) that did not previously exist.

Use the names as they appear on attack.mitre.org today, and if you are reading an older report that says Defense Evasion, understand it means the same thing. Appendix A.1 lists the current tactic names.

9.2 Looking up a technique

1. Go to attack.mitre.org. The front page shows the Enterprise matrix: tactics across the top, techniques listed underneath each one.
2. To go straight to a technique, use the address attack.mitre.org/techniques/T1190/ and swap in the identifier you want.
3. For a sub-technique, the identifier splits across the path. T1505.003 is at attack.mitre.org/techniques/T1505/003/. This catches everyone once.
4. On the technique page, read the description and check it genuinely matches what you observed. Record the identifier, the official name and the tactic.
5. Read the **Detection** section near the bottom of the page. It describes what data sources reveal this technique. You will use this in Section 10 when you write your rules.

9.3 Build the mapping

Below is the mapping, partly filled in. Complete the empty rows from your Day 3 and Day 4 evidence. Every row needs a piece of evidence you can point at.

Tactic	Technique ID	Technique name	Your evidence
Reconnaissance	T1595.002	Vulnerability Scanning	Egress log, 28 Aug 01:52, 214 sessions to port 80 carrying zero bytes
Initial Access	T1190	Exploit Public-Facing Application	MTB-WEB01 — 02 Sep 02:12:33 UTC — Apache access log shows encoded

Tactic	Technique ID	Technique name	Your evidence
			/cgi-bin/ path traversal successfully returning /etc/passwd.
Persistence	T1505.003	Web Shell	MTB-WEB01 — 02 Sep 02:15:22 UTC — style.php written into the web upload directory; first POST to the webshell occurred at 02:17:40.
Privilege Escalation	T1068	Exploitation for Privilege Escalation	MTB-WEB01 — 02 Sep 02:21:05 UTC — pkexec opened a root session for www-data, matching the vulnerable policykit package.
Persistence	T1136.001	Local Account	MTB-WEB01 — 02 Sep 02:23:11 UTC — svc-backup created with UID 0. MTB-FILE01 — 02:54:39 — same UID 0 backdoor account created.
Persistence	T1098.004	SSH Authorized Keys	MTB-WEB01 — 02 Sep 02:24:48 UTC — attacker SSH key added to /root/.ssh/authorized_keys.
Persistence	T1053.003	Cron	MTB-WEB01 — 02 Sep 02:26:30 UTC — /etc/cron.d/apache-logrotate created; system log records cron reload at 02:26:31.
Persistence	T1543.002	Systemd Service	MTB-FILE01 — 02 Sep 02:56:03 UTC — system log records Started System Log Forwarder, which launches the malicious beacon through systemd.
Credential Access	T1552.004	Private Keys	MTB-WEB01 / MTB-FILE01 — by 02 Sep 02:52:14 UTC — FILE01 accepted public-key SSH from WEB01, showing the attacker used SSH private-key material available on the compromised web server.
Lateral Movement	T1021.004	SSH	MTB-FILE01 — 02 Sep 02:52:14 UTC — accepted public-key SSH login for svc-backup from MTB-WEB01 (10.50.2.24).
Collection	T1560.001	Archive via Utility	MTB-WEB01 — 02 Sep 02:34:55 UTC — customer data archived into /tmp/.ICE-unix; recovered bash history shows tar czf used.
Command and Control	T1071.001	Web Protocols	MTB-WEB01 — 02 Sep 02:31:12 UTC — outbound C2 session on 8443. MTB-FILE01 — 02:57:40 — outbound C2 session on 443. Source: perimeter firewall telemetry.

Tactic	Technique ID	Technique name	Your evidence
Exfiltration	T1041	Exfiltration Over C2 Channel	MTB-WEB01 — 02 Sep 02:38:02 UTC — 2.88 MB outbound over its C2 channel. MTB-FILE01 — 05:12:19 — 9.92 MB outbound over its C2 channel.
Stealth	T1070.003	Clear Command History	MTB-WEB01 — 02 Sep 02:41:19 UTC — shell history cleared; recovered root history contains history -c.
Stealth	T1027	Obfuscated Files or Information	MTB-WEB01 — 02 Sep 02:15:22 UTC — recovered style.php contained a Base64-encoded payload string that was later decoded in CyberChef.

9.4 Step by step

1. Complete the evidence column for all fifteen rows. Each entry needs a host, a timestamp and a source.
2. Two techniques in the table appear on both compromised hosts but with different implementations. Identify them and note both implementations.

T1071.001 — Web Protocols: MTB-WEB01 used apache-worker with outbound C2 on TCP 8443, while MTB-FILE01 used syslog-forward with outbound C2 on TCP 443.

T1041 — Exfiltration Over C2 Channel: MTB-WEB01 sent about 2.88 MB through its C2 channel, while MTB-FILE01 sent about 9.92 MB using different C2 infrastructure.

3. One technique in the table is evidenced by something you decoded rather than something you observed running. Identify it.

T1027 — Obfuscated Files or Information. The recovered style.php contained a Base64-encoded string, which was decoded during the Day 3 CyberChef exercise.

4. Add at least two techniques that are not in the table but that you have evidence for. Appendix A.1 lists the full set used in this incident if you want to check your work afterwards - try it yourself first.

Tactic	Technique ID	Technique	Evidence
Initial Access / Persistence / Privilege Escalation / Stealth	T1078.003	Local Accounts	MTB-FILE01 — 02 Sep 02:52:14 UTC — attacker successfully authenticated using the svc-backup local account created as UID 0.

Collection	T1005	Data from Local System	MTB-WEB01 — 02 Sep 02:34:55 UTC — local customer data under /var/www/html/data was collected and archived for staging.
-------------------	--------------	-------------------------------	---

5. For each of the fifteen ATT&CK tactics, note whether this attacker used it. The ones they did not use are as informative as the ones they did.

Reconnaissance = Used - Vulnerability scanning against MTB-WEB01 was observed in the perimeter firewall logs.

Resource Development = No evidence - The available evidence does not show the attacker preparing infrastructure, accounts, or other resources beforehand.

Initial Access = Used - The attacker exploited the public-facing Apache server.

Execution = Used - Commands were executed through the webshell and scheduled execution mechanisms such as cron were used.

Persistence = Used - The attacker used a webshell, UID 0 account, SSH authorized key, cron job, and systemd service.

Privilege Escalation = Used - pkexec was used to obtain root privileges on MTB-WEB01.

Stealth = Used - Shell history was cleared and obfuscated/Base64-encoded content was used.

Credential Access = Used -SSH private-key material from MTB-WEB01 was used to support access to MTB-FILE01

Discovery = Used - Commands and evidence showed host, account, file, and network discovery activity

Lateral Movement = Used - SSH was used to move from MTB-WEB01 to MTB-FILE01.

Collection = Used - Local customer data was gathered and archived before exfiltration.

Command and Control = Used - Both compromised hosts maintained outbound communications with attacker-controlled infrastructure.

Exfiltration = Used - Large outbound transfers occurred over the C2 channels from both compromised hosts.

Impact = No evidence - No destructive, disruptive, or availability-impacting activity was identified.

Defense Impairment = No evidence - No evidence shows security tools or defensive controls being disabled or degraded.

The test of a good mapping

Hand your completed table to someone who has not read your reports. They should be able to describe the intrusion from it alone, in order, without asking you a question.

If they cannot, you have mapped labels rather than an attack.

10. Day 5, Part 2: Writing Detections

Your Day 3 report recommended better monitoring. Your Day 4 hunt showed exactly why the obvious detections would not have worked: block one address and the attacker uses another, hash one file and the next one has a different name.

This section turns the recommendation into something a SOC can actually deploy.

10.1 What Sigma is

Sigma is an open, vendor-neutral format for writing detection rules. A rule is a small YAML file describing what to look for. Converters then translate that one rule into a Splunk search, a Sentinel KQL query, an Elastic query or whatever the organisation runs.

This matters because detection logic written directly in a vendor's query language is locked to that vendor. Meridian Trust has changed SIEM twice in six years. The Sigma rules survived both migrations; the hand-written searches did not.

10.2 The parts of a rule

Field	Required?	What it holds
title	Mandatory	A short description of what the rule detects. Keep it under about 50 characters.
logsource	Mandatory	Which logs to run against. Built from category, product and service.
detection	Mandatory	What to match. Contains one or more named search identifiers plus a condition.
condition	Mandatory	Inside detection. How the named searches combine, for example 'selection and not filter'.
id	Optional	A UUID so the rule can be referenced. Generate one or leave it out for this lab.
status	Optional	One of stable, test, experimental, deprecated, unsupported.
description	Optional	Fuller explanation, including why the rule exists.
references	Optional	Links justifying the rule, for example the ATT&CK technique page.
author	Optional	Who wrote it.
date	Optional	ISO 8601 format, YYYY-MM-DD. Not any other format.
tags	Optional	Namespaced and lower case, for example attack.t1505.003.
falsepositives	Optional	What legitimate activity could also match. Never leave this empty in practice.
level	Optional	One of informational, low, medium, high, critical.

Two rules about YAML that will cost you time if you ignore them

Indentation is meaningful and must be spaces, never tabs. Four spaces per level is the Sigma convention. If your editor inserts tabs, turn that off before you start.

A colon followed by a space separates a key from its value. A colon inside a value, such as in a file path or a URL, must be inside quotes or YAML will try to read it as another key.

10.3 A complete worked rule

This is a full, valid Sigma rule detecting the backdoor account creation you found on both compromised hosts. Read every line before you write your own.

```

title: Linux Local Account Created With UID 0
id: 8f2c1a44-3b7e-4d19-9c02-5ae6f1d0b731
status: experimental
description: |
    Detects creation of a Linux local account assigned UID 0. Such an account
    holds full root privilege while appearing in /etc/passwd as an ordinary
    service account, and useradd logs a warning when it happens.
    Observed on MTB-WEB01 and MTB-FILE01 during case MTB-IR-2026-0902.
references:
  - https://attack.mitre.org/techniques/T1136/001/
author: Meridian Trust Bank SOC
date: 2026-09-04
tags:
  - attack.t1136.001
  - attack.t1078.003
logsource:
  product: linux
  service: auth
detection:
  keywords:
    - 'UID 0 is already in use'
  condition: keywords
falsepositives:
  - Deliberate creation of a second root-equivalent account under a documented
    break-glass procedure. Should be rare and covered by a change record.
level: critical

```

10.4 Why that rule is a good one

- **It sits high on the Pyramid.** It does not name an account, an address or a file. Any attacker creating a UID 0 account on any host trips it. To evade it they must stop using that technique.
- **It fires on the attacker's own noise.** The operating system generates that warning by itself. The attacker cannot suppress it without also failing to create the account.
- **Its false positive case is named and realistic.** A break-glass account is a real thing that would match. Saying so tells the SOC how to triage a hit rather than leaving them guessing.
- **It would have fired twice in this incident,** at 02:23:11 on MTB-WEB01 and again at 02:54:39 on MTB-FILE01, the second one 53 minutes before any alert actually reached a human.

10.5 Write four rules

Write one rule per numbered requirement, saved as a separate .yaml file. Use the worked rule as your template.

Rule	What it must detect	Evidence it must catch
title	Mandatory	A short description of what the rule detects. Keep it under about 50 characters.
logsource	Mandatory	Which logs to run against. Built from category, product and service.
detection	Mandatory	What to match. Contains one or more named search identifiers plus a condition.

Rule	What it must detect	Evidence it must catch
condition	Mandatory	Inside detection. How the named searches combine, for example 'selection and not filter'.
id	Optional	A UUID so the rule can be referenced. Generate one or leave it out for this lab.
status	Optional	One of stable, test, experimental, deprecated, unsupported.
description	Optional	Fuller explanation, including why the rule exists.
references	Optional	Links justifying the rule, for example the ATT&CK technique page.
author	Optional	Who wrote it.
date	Optional	ISO 8601 format, YYYY-MM-DD. Not any other format.
tags	Optional	Namespaced and lower case, for example attack.t1505.003.
false positives	Optional	What legitimate activity could also match. Never leave this empty in practice.
level	Optional	One of informational, low, medium, high, critical.

10.6 Requirements for every rule you write

1. Include all four mandatory fields, plus description, tags, falsepositives and level. A rule with no falsepositives entry will not be accepted.
2. Tag it with at least one ATT&CK technique from your Section 9 mapping, in the form `attack.t1053.003`. Lower case, dots as separators.
3. Write the description so it says what the rule detects and why it exists. Reference the case number.
4. Set the level honestly. Not everything is critical. A rule that fires on something needing review this week is medium, not critical.
5. In the falsepositives field, name at least one specific legitimate activity that would match. If you cannot think of one, your rule is probably too broad and you have not thought about it hard enough.
6. Below each rule, in your report, write two sentences: which Pyramid tier it targets, and what the attacker would have to change to evade it.

title: Persistence Executing From Usr Local

status: experimental

description: |

Detects cron or systemd launching an executable from /usr/local/.

This behavior was observed on compromised systems during case MTB-IR-2026-0902. Assumes process creation telemetry records the executable path and parent process.

tags:

- attack.t1053.003
- attack.t1543.002

logsource:

product: linux
category: process_creation

detection:**selection_path:**

Image|startswith:

- '/usr/local/bin/'
- '/usr/local/sbin/'

selection_parent:

ParentImage|endswith:

- '/cron'
- '/crond'
- '/systemd'

condition: selection_path and selection_parent

falsepositives:

- Legitimate locally installed administrative, monitoring, or backup software launched by an approved cron job or systemd service.

level: high

Pyramid tier: Tier 6 because the rule detects persistence behavior rather than a specific IP address, hash, or malicious filename.

Evasion: To evade this detection, the attacker would need to change the persistence mechanism, execute the payload from somewhere outside /usr/local/, or otherwise change the behavior instead of simply renaming the file.

The most common mistake on this section

Writing a rule that matches 185.220.101.47. It will detect this incident perfectly and nothing else ever again.

Before you save each rule, ask: if the attacker came back tomorrow from a different address, with different filenames, would this still fire? If the answer is no, rewrite it.

That question is the entire discipline of detection engineering.

10.7 Record: your detection set

Rule file	What it detects	ATT&CK tag	Level	Pyramid tier
mtb_detect_01	Cron or systemd persistence executing a binary from /usr/local/	attack.t1053.003, attack.t1543.002	High	Tier 4
mtb_detect_02	A script file written into a web-accessible upload directory	attack.t1505.003	High	Tier 6
mtb_detect_03	Encoded directory traversal in a request against a CGI path	attack.t1190	High	Tier 6
mtb_detect_04	Accepted SSH login for a service account coming from another server-segment host	attack.t1021.004, attack.t1078.003	High	Tier 6

11. Day 5, Part 3: Containment, Eradication and Recovery

Two hosts are compromised and you know how. The bank now needs them back in service, safely, and it needs to be certain the attacker is gone rather than hoping so.

11.1 The three phases, and why the order matters

Phase	Goal	Gets this wrong how
Containment	Stop the attacker acting, without destroying evidence or tipping them off before you are ready.	Rebuilding a host immediately, which removes the attacker and every trace of what they did.
Eradication	Remove every access path they established, not just the one you found first.	Deleting the webshell and declaring victory while three other persistence mechanisms remain.
Recovery	Return to normal service with confidence, and watch for their return.	Restoring from a backup taken after the compromise, which restores the compromise.

The backup question you must answer in your plan

The intrusion began on 28 August. Any backup of MTB-WEB01 taken after the first successful exploitation may contain attacker artefacts.

Your recovery plan has to state which backup generation is safe to restore from, and how you would verify that before trusting it.

Teams that skip this restore the incident along with the service and then spend a fortnight wondering how the attacker got back in.

11.2 Build the containment plan

1. List every containment action for both compromised hosts, in the order you would perform them. For each, state what it stops and what evidence it risks destroying.

- Decide whether to power the hosts off, isolate them at the network, or leave them running under observation. Justify the choice using what you learned about order of volatility on Day 3.

MTB-WEB01 and MTB-FILE01 should be isolated at the network rather than immediately powered off. Powering them off would destroy volatile evidence such as memory contents, active network sessions, running processes, and logged-in sessions. After volatile evidence and forensic images are captured, network isolation provides containment while preserving the systems for further investigation.

- Identify what containment must happen on hosts that are not compromised. The attacker had a private key; the exposure is not limited to the two hosts they used it on.

All other hosts should be checked for trust of the exposed SSH key, and that key should be revoked everywhere it is accepted. Authentication logs should be reviewed for connections from MTB-WEB01 or MTB-FILE01, and server-to-server SSH should be restricted to approved management paths. The known attacker destinations should be blocked environment-wide, and any new host matching the four detection rules should be investigated immediately.

11.3 Build the eradication plan

List every access path the attacker established. There are more than most people count on the first attempt. Work through both hosts systematically.

Access path	Host(s)	How you remove it	How you verify removal
Webshell in upload directory	MTB-WEB01	Remove the malicious webshell after preserving a forensic copy and evidence.	Search the web root/upload directories for scripts and run the web-upload detection rule.
svc-backup UID 0 account	MTB-WEB01 , MTB-FILE01	Remove the account and any associated home directory/credentials after evidence collection.	Review /etc/passwd, /etc/shadow, UID 0 accounts, and authentication logs to confirm only approved root-equivalent accounts remain.
Attacker SSH authorized key	MTB-WEB01 and any trusting systems	Remove the unauthorized key and rotate affected SSH keys.	Review all authorized_keys files and attempt validation using the replacement credentials only.
Compromised SSH private-key trust	Environment-wide	Revoke the exposed private key and replace the corresponding trusted public key everywhere.	Inventory every host that trusted the old key and confirm the old credential can no longer authenticate.
Cron persistence and malicious beacon	MTB-WEB01	Remove the malicious cron entry and dropped executable after preserving copies.	Review cron locations, running processes, and apply mtb_detect_01 to confirm no persistence from /usr/local/.
Systemd persistence and malicious beacon	MTB-FILE01	Disable/remove the unauthorized System Log Forwarder service and	Review enabled systemd units, running services, and

Access path	Host(s)	How you remove it	How you verify removal
		malicious executable after evidence collection.	process/network telemetry for recurrence.
Apache exploitation path and configuration	MTB-WEB01	Patch/upgrade the vulnerable Apache installation and correct the dangerous ScriptAlias / Require all granted configuration.	Confirm patched version, review Apache configuration, and test that encoded traversal requests are rejected.
PwnKit privilege-escalation vulnerability	MTB-WEB01 and any similarly vulnerable Linux hosts	Update the vulnerable policykit/pkexec package to a fixed version.	Verify package versions and confirm the vulnerable configuration/version is no longer present.

1. Complete the table. Include everything: accounts, keys, scheduled execution, services, dropped files, and the vulnerabilities that let them in.
2. Identify every credential and key that must be treated as compromised, and state the blast radius of each. For the SSH private key, that means every host that trusted it, not just the two you found it used on.

The svc-backup account must be treated as attacker-controlled on both compromised hosts and removed after evidence collection. The SSH private key exposed on MTB-WEB01 must be considered compromised everywhere it was trusted; therefore, every server accepting that key must have the trust removed and the key pair rotated. Any administrative or service credentials accessible to root on MTB-WEB01 or MTB-FILE01 should also be reviewed and rotated because root access could have exposed them.

3. State how you would confirm eradication is complete. Your Section 10 detection rules are part of the answer.

Eradication is complete only after both systems and the wider server segment are re-hunted. Authentication logs should show no attacker-controlled accounts or keys, cron and systemd persistence should be absent, malicious binaries and webshells should no longer exist, and outbound network telemetry should show no unexplained C2 behavior. All four Sigma detection rules from Section 10 should be run against the restored systems and recent logs with no unexplained matches.

11.4 Build the recovery plan

1. State the order in which the two hosts return to service, and why that order.

MTB-FILE01 should return to service first, because it is an internal server and can be validated under controlled network access before restoring the internet-facing system. Once FILE01 operates normally and monitoring shows no recurrence, MTB-WEB01 should return last, because exposing the public web server again creates the greatest risk of immediate re-exploitation.

2. State which backup generation is safe, based on your revised start of intrusion, and how you would verify it before restoring.

A backup from before August 30, 2026 should be used, preferably the most recent verified backup from August 29 or earlier. The backup should be mounted and inspected offline before restoration. Verify that it does not contain the webshell, attacker accounts or keys, malicious cron/systemd persistence, or dropped beacon files, and confirm system/package versions before trusting it.

3. State what must be true before MTB-WEB01 is exposed to the internet again. Both vulnerabilities from your Day 1-2 assessment are part of this, and so is the Apache configuration.
 - **Apache must be upgraded so the known path-traversal/RCE vulnerabilities are no longer present.**
 - **The policykit/pkexec privilege-escalation vulnerability must be patched.**
 - **The dangerous Apache ScriptAlias plus overly broad Require all granted configuration must be corrected.**
 - **The attacker-created account, SSH keys, cron persistence, webshell, and beacon must be removed.**
 - **Exposed credentials and SSH keys must be rotated.**
 - **The host must pass vulnerability validation and all four detection-rule checks.**
 - **Network egress should be restricted to approved destinations/services.**
4. Define a heightened monitoring period: how long, watching for what, and using which of your four detection rules.

For 30 days after recovery, the SOC will apply heightened monitoring to both restored hosts and the surrounding server segment. Analysts will review outbound connections, unusual server-to-server SSH authentication, new cron or systemd persistence, scripts written into web upload directories, and encoded CGI traversal attempts.

Attackers come back

They know the environment, they know what worked, and they often still hold a credential nobody thought to rotate. A recovery plan that ends the moment service is restored is only half a plan. Say explicitly what is watched, for how long, and who is watching it.

12. Day 5, Part 4: Post-Incident Review

The last deliverable is the one that decides whether any of this work matters beyond this incident. A post-incident review asks what let this happen and what changes so that it does not happen the same way twice.

12.1 The detection gap

Work this out precisely, because it is the core finding of the review.

Moment	Timestamp	How long after first activity
First attacker activity	2026-08-28 01:52:17 UTC	0
First successful exploitation	2026-08-30 02:07:55 UTC	2 days, 15 min, 38 sec
Second host compromised	2026-09-02 02:52:14 UTC	5 days, 59 min, 57 sec
Largest data transfer	2026-09-02 05:12:19 UTC	5 days, 3 hr, 20 min, 2 sec
Alert reached a human	2026-09-02 09:47:12 UTC	5 days, 7 hr, 54 min, 55 sec

1. Fill in the table from your extended timeline.
2. For each of your four detection rules from Section 10, state when it would have fired. Then state how much of the intrusion would have been prevented if the SOC had acted on that first firing.

mtb_detect_01 — Persistence from /usr/local/

It would first fire on MTB-WEB01 at 02:26:30 on September 2, when the malicious cron persistence was created. If the SOC immediately isolated the host, the attacker already had root, a UID 0 account, and an SSH key, but the response could likely have prevented the large WEB01 transfer, lateral movement to FILE01, and the later FILE01 exfiltration.

mtb_detect_02 — Script written into web upload directory

Would fire at 02:15:22 on September 2, when style.php appeared in the upload directory. Immediate response could have occurred before privilege escalation at 02:21:05, likely preventing the root compromise, persistence, lateral movement, and both major outbound transfers.

mtb_detect_03 — Encoded CGI directory traversal

It would first fire on August 30 at approximately 02:07:55 UTC, when the successful encoded traversal occurred. This is the earliest of the four detections and could have stopped the incident before the attacker established persistent access or returned on September 2. The lab specifically requires this rule to detect the encoded CGI traversal seen on August 30 and September 2.

mtb_detect_04 — Server-to-server service-account SSH login

Would fire at 02:52:14 on September 2, when MTB-FILE01 accepted the svc-backup public-key SSH login from MTB-WEB01. It would not prevent the initial WEB01 compromise or the lateral movement event itself, but rapid containment could have prevented the new UID 0 account, systemd persistence, beaconing, and the 9.92 MB FILE01 transfer.

3. Identify which single rule would have made the biggest difference, and say why.

mtb_detect_03 would have made the biggest difference because it would have detected the exploitation attempt on August 30, before the attacker established persistent access and several days before the SOC alert. Acting on that alert could have prevented the later root compromise, lateral movement to MTB-FILE01, and both major data transfers.

4. Note honestly what none of your rules would have caught. Every detection set has gaps, and a review that claims otherwise is not credible.

The four rules do not cover every stage of the attack. They would not directly detect the original vulnerability scanning on August 28, the pkexec privilege escalation, creation of the UID 0 account, installation of an SSH authorized key, all C2 beaconing patterns, or data exfiltration itself. Additional detections are still required for those behaviors.

12.2 The uncomfortable questions

A post-incident review that only discusses the attacker has failed. Answer all four of these, plainly.

- **Why was a host with a publicly known, actively exploited vulnerability still exposed to the internet in September**, when the assessment on 8 September flagged it and it had been in the KEV catalogue since 2021? Your answer must address the patching process, not the attacker.

The main failure was the vulnerability and patch-management process. A known, actively exploited vulnerability on an internet-facing system should have triggered an urgent remediation deadline instead of remaining exposed under a normal patch schedule. Internet-facing KEV vulnerabilities should have defined emergency SLAs, tracked exceptions, and verification that remediation was actually completed.

- **Why did the assessment run on 8 September, after the first successful exploitation on 30 August?** What scan cadence would have caught this in time, and what would that cost?

The vulnerability-assessment cadence was too slow for a critical internet-facing system. A daily or continuous external vulnerability scan of internet-facing assets would have had a much better chance of identifying the exposure before August 30, while broader internal assessments could remain weekly or scheduled. The cost would include additional scanning resources, more analyst time for validation, and more remediation workload, but that cost is significantly lower than responding to a multi-host compromise and data loss.

- **Why was an unattended SSH private key trusted by other hosts sitting on an internet-facing server in the DMZ?** What should the trust relationship between the DMZ and the server segment look like?

The network trust model allowed an internet-facing DMZ server to hold credentials that were trusted by systems deeper in the environment. The DMZ should not have unrestricted SSH trust into the server segment. Administrative access should instead use a controlled bastion or management path, least privilege, short-lived credentials where possible, and tightly restricted firewall rules so compromise of a public server does not automatically provide a path inward.

- **Why did seven and a half hours of five-minute beaconing to a non-allowlisted destination not alert anyone sooner?** The egress log recorded every one of those connections. The data existed. What was missing?

The problem was not missing telemetry; the firewall recorded the connections. What was missing was detection logic, correlation, and alert routing that recognized repeated outbound connections to a non-approved destination as suspicious. The fix is to create SIEM detections for periodic outbound traffic to non-allowlisted destinations, combine them with asset and allowlist context, and ensure high-confidence alerts are routed to the SOC for timely investigation. The lab specifically points out that the data existed for the entire beaconing period but nobody acted on it

Write this without blaming a person

Every one of those questions has an answer that is about a process, a control or a priority. None of them has a useful answer that is about an individual being careless.

Reviews that name individuals stop people reporting things. Within a year the organisation is less safe than before, because the next person who notices something odd decides not to mention it.

Write about what the system allowed, not about who did it.

12.3 Report structure to follow

Section	Contents
Executive summary	Meridian Trust experienced a multi-host intrusion that began with attacker activity against the public web server on August 28 and progressed to successful compromise by August 30. The attacker obtained privileged access to MTB-WEB01, established persistence, moved laterally to MTB-FILE01, and generated two suspicious outbound transfers totaling approximately 12.8 MB. The investigation identified weaknesses in vulnerability management, network trust, and monitoring rather than a single individual failure. The primary improvements are faster remediation of actively exploited vulnerabilities, stronger DMZ segmentation, improved credential controls, and behavioral SOC detections.
What happened	The attacker first scanned the public-facing web server, later exploited the web application, and established persistent access. They gained root privileges, created privileged access mechanisms, installed command-and-control tooling, and used trusted SSH access to move from MTB-WEB01 into MTB-FILE01. Different persistence and C2 methods were used on the two hosts, demonstrating why exact indicator matching alone was insufficient. Suspicious outbound transfers occurred from both compromised systems before the activity was detected and investigated.
ATT&CK mapping	Your completed table from Section 9.
Detection gap	The timeline table from Section 12.1 and what each rule would have changed.
New detections	Your four Sigma rules, with the Pyramid tier and evasion 10.7
Containment, eradication, recovery	Section 11 plan, including the decision to restore WEB01 only from a verified pre-compromise backup
Root cause and contributing factors	four Section 12.2 answers above.
Recommendations	● Establish an emergency remediation SLA for internet-facing KEV vulnerabilities. ● Run daily or continuous external vulnerability scans on internet-facing systems. ● Remove direct SSH trust from DMZ systems into internal servers. ● Rotate the exposed SSH keys and review every host that trusted them. ● Deploy the four new behavioral Sigma detection rules. ● Alert on repeated outbound traffic to non-allowlisted destinations. ● Review DMZ segmentation and privileged credential storage. ● Maintain a 30-day heightened monitoring period after recovery.

13. Deliverables

Submit the following through the class shared drive.

13.1 Day 4

- Your extended timeline, merged with the Day 3 timeline, covering the full intrusion.
- Your completed fleet hunt grid, including the columns you added for newly discovered indicators.
- Your tiered indicator list, with a Pyramid tier and an attacker-cost note for every entry.
- Your three behaviour descriptions that name no address, filename or hash.
- Your incident scoping report, answering how far back, how many hosts, what data, with a stated dwell time and named intelligence gaps.

13.2 Day 5

- Your completed ATT&CK mapping, with evidence for every row and at least two techniques you added yourself.
- Four Sigma rule files, each with all mandatory fields plus description, tags, falsepositives and level.
- For each rule, the Pyramid tier it targets and what an attacker would change to evade it.
- Your containment, eradication and recovery plan, including the backup generation decision and the credential blast radius.
- Your post-incident review, including the detection gap table and answers to all four questions in Section 12.2.

14. Reflection Questions

Answer briefly in your own words, three to five sentences each. Graded on reasoning, not length.

14.1 Day 4

1. Your Day 3 forensic report gave a start date that turned out to be wrong by several days. The investigation was not careless. Explain what made the disk evidence misleading, and what that tells you about relying on any single evidence source.

The disk evidence was misleading because the Apache logs had already rotated, so older activity was no longer visible on the host. The perimeter firewall kept longer-term telemetry outside the compromised server, which revealed activity going back several days earlier. This shows that relying on only one evidence source can give you an incomplete picture of an incident.

2. The attacker used a different address, filename and persistence mechanism on the second host. What does that tell you about how they operate, and what would you now assume about a third host if you found one?

It shows that attackers can easily change IP addresses, filenames, and persistence methods to avoid simple indicator-based detection. If I found a third compromised host, I would not assume it would use the same exact IOCs. I would hunt for the same behaviors and techniques instead.

3. MTB-DVWA01 showed the same beacon-like periodicity that got MTB-WEB01 flagged, and it was benign. Explain what actually distinguished them, and why an analyst who cannot make that distinction is a problem for their team.

MTB-DVWA01 looked suspicious because it connected externally on a regular five-minute interval, but the asset register showed that this was expected behavior from an approved monitoring agent. The process, destination, and timing all matched the documented configuration. An analyst who cannot tell the difference between normal and malicious behavior will create too many false positives and eventually cause people to ignore alerts.

4. Why is documenting what you checked on the clean hosts as important as documenting what you found on the compromised ones?

Documenting what was checked shows why you believe a host is clean instead of just saying that nothing was found. It also makes it clear which evidence sources and indicators were reviewed. That gives other analysts and leadership confidence in the conclusion and shows where any remaining blind spots may exist.

14.2 Day 5

1. Explain the Pyramid of Pain in your own words, using an example from this incident, and say why blocking the C2 address after Day 3 achieved so little.

The Pyramid of Pain shows that some indicators are much easier for attackers to change than others. For example, blocking the first C2 IP did not help much because the attacker was already using a different address on MTB-FILE01. Detecting the attacker's behavior, such as persistent outbound communication launched through a persistence mechanism, creates more difficulty because they have to change how they operate rather than just change an IP.

2. The account-creation rule in Section 10.3 detects the attacker using a warning the operating system produced by itself. Why is a detection built on the target's own logging harder to evade than one built on the attacker's tooling?

A detection based on the operating system's own logging is harder to evade because the attacker does not fully control how the system records certain actions. In this case, creating a second UID 0 account caused Linux itself to generate a warning. An attacker can easily rename a tool or change a file hash, but avoiding the system-generated evidence may require changing the technique entirely.

3. Every one of your rules will produce false positives eventually. Explain why the false positives field matters more to the analyst receiving the alert than to the person writing the rule.

The analyst receiving the alert needs to quickly decide whether the activity is actually malicious or something legitimate. A good false-positive description gives them context about normal activity that could trigger the same rule. That saves investigation time and helps prevent real alerts from getting lost in unnecessary noise.

4. The egress log recorded all seven and a half hours of beaconing. The data existed and nobody acted on it. What does that suggest about where security programmes actually fail, and how would you fix it here?

It shows that simply collecting security data is not enough if nobody has detection logic in place to recognize important patterns. The firewall had the evidence, but there was no effective alerting or correlation to bring it to the SOC's attention. I would fix this by creating behavioral detections for repeated outbound communication to non-approved destinations and making sure high-confidence alerts are routed to analysts quickly.

15. Grading Alignment

This lab counts toward your lab-completion and documentation-quality grading criteria.

For Day 4, the marks are in your scoping discipline. Anyone can find the compromised host once they are told to look. What is graded is whether a reader can tell what you checked, what you did not check, and how confident you are in each conclusion. A report that states certainty it has not earned will score below one that names its gaps.

For Day 5, the marks are in whether your detections would survive the attacker changing their infrastructure. Four rules that match addresses and hashes will score poorly however well formatted they are. Two good behavioural rules with honest false positive notes are worth more than four brittle ones.

Across both, your reports are graded on whether someone outside the incident could act on them. The executive summaries are for a person, not a scanner.

16. Completion Checklist

Work down this list before you submit. Every line is something that is graded. If you cannot tick a line, that is where to spend your remaining time.

16.1 Day 4

Done	Item
	Egress log copied into a spreadsheet and split into columns
	Revised start of intrusion identified, with the exact log row cited
	Difference between the revised start and the Day 3 start calculated in days
	Written explanation of why the disk image could not show the earlier activity
	All three external destination addresses identified
	Both large outbound transfers recorded, with host, timestamp and volume in megabytes
	Fleet hunt grid completed for all five hosts, every cell answered
	Second compromised host identified, with the SSH login timestamp and account
	Explanation of how the attacker authenticated to the second host
	All three tradecraft differences on the second host noted
	New indicator columns added to the grid and every host re-checked
	MTB-DVWA01 verdict reached, with the specific evidence that settles it
	For each clean host, the indicators and log sources checked are listed
	MTB-APP01 failed logins assessed and the judgement justified
	All sixteen or more indicators assigned a Pyramid tier
	Attacker cost to change recorded for every tier 1 and tier 2 indicator
	Three behaviour descriptions written that name no address, filename or hash
	Scoping report written with all seven sections from Section 8.2
	Dwell time calculated and the working shown
	Intelligence gaps named

16.2 Day 5

Done	Item
	All fifteen mapping rows completed with host, timestamp and source
	The two techniques implemented differently on each host identified
	The technique evidenced by decoding rather than observation identified

Done	Item
	At least two additional techniques added with evidence
	All fifteen tactics assessed for whether the attacker used them
	Four Sigma rule files written and saved separately
	Every rule has title, logsource, detection and condition
	Every rule has description, tags, falsepositives and level
	Every rule tagged with at least one ATT&CK technique in attack.tXXXX form
	Every falsepositives field names a specific realistic legitimate activity
	No rule matches an IP address, filename or hash as its only logic
	Pyramid tier and evasion note written for each rule
	Containment actions listed in order, with what each stops and what it risks
	Power-off versus isolate decision made and justified using order of volatility
	Containment actions for non-compromised hosts identified
	Eradication table completed with every access path on both hosts
	Credential and key blast radius stated, including every host trusting the SSH key
	Verification method for eradication stated
	Recovery order stated and justified
	Safe backup generation identified, based on the revised start of intrusion
	Conditions for re-exposing MTB-WEB01 to the internet stated
	Heightened monitoring period defined, with duration and which rules
	Detection gap table completed with all five timestamps
	Firing time and prevented impact stated for each of the four rules
	The single highest-value rule identified and justified
	What none of the rules would catch stated honestly
	All four questions in Section 12.2 answered, about process not people
	Post-incident review written with all eight sections from Section 12.3

Before you submit

Read both executive summaries out loud. If you stumble, or if you used a word a branch manager would not know, rewrite them.

Open each Sigma rule and ask the evasion question one more time: different address, different filenames, would it still fire?

Check that no live, clickable external address survives anywhere in your indicator list or your reports.

Appendix A: Reference Data and Troubleshooting

Everything in this appendix is here so that nothing can stop you. If a website is slow, blocked or down, the values you would have looked up are printed below. Use them, and note in your report that you used the printed reference rather than the live source.

A.1 MITRE ATT&CK reference

Current Enterprise tactics, in the order an intrusion typically moves through them. Note the names marked as changed - older material uses different words for the same thing.

ID	Tactic	Note
TA0043	Reconnaissance	
TA0042	Resource Development	
TA0001	Initial Access	
TA0002	Execution	
TA0003	Persistence	
TA0004	Privilege Escalation	
TA0005	Stealth	Renamed. Older material calls this Defense Evasion.
TA0006	Credential Access	
TA0007	Discovery	
TA0008	Lateral Movement	
TA0009	Collection	
TA0011	Command and Control	
TA0010	Exfiltration	
TA0040	Impact	
TA0112	Defense Impairment	Newer tactic. Not present in older material.

Techniques relevant to case MTB-IR-2026-0902. Identifiers and names as published by MITRE. Do the lookups yourself first - this table is a safety net and a way to check your work, not a shortcut past the exercise.

Technique ID	Official name	Tactic
T1595.002	Vulnerability Scanning	Reconnaissance
T1190	Exploit Public-Facing Application	Initial Access

Technique ID	Official name	Tactic
T1505.003	Web Shell	Persistence
T1068	Exploitation for Privilege Escalation	Privilege Escalation
T1136.001	Local Account	Persistence
T1098.004	SSH Authorized Keys	Persistence, Privilege Escalation
T1053.003	Cron	Execution, Persistence, Privilege Escalation
T1543.002	Systemd Service	Persistence, Privilege Escalation
T1078.003	Local Accounts	Stealth, Persistence, Privilege Escalation, Initial Access
T1552.004	Private Keys	Credential Access
T1021.004	SSH	Lateral Movement
T1046	Network Service Discovery	Discovery
T1033	System Owner/User Discovery	Discovery
T1083	File and Directory Discovery	Discovery
T1087.001	Local Account	Discovery
T1005	Data from Local System	Collection
T1560.001	Archive via Utility	Collection
T1071.001	Web Protocols	Command and Control
T1105	Ingress Tool Transfer	Command and Control
T1041	Exfiltration Over C2 Channel	Exfiltration
T1070.003	Clear Command History	Stealth
T1027	Obfuscated Files or Information	Stealth

A.2 Sigma reference

Allowed values for the fields that only accept a fixed set. Getting these wrong is the most common reason a rule fails validation.

Field	Allowed values
status	stable, test, experimental, deprecated, unsupported
level	informational, low, medium, high, critical
logsource	Any combination of category, product, service and definition
date and modified	ISO 8601 with separators only: YYYY-MM-DD
tags	Lower case, dot-separated namespaces, for example attack.t1053.003

A second complete rule, in a different shape from the one in Section 10.3, showing a named selection with a filter and a compound condition.

```

title: Outbound Session From Unmanaged Root Binary
id: 3d5b90e7-11ac-4c68-8b25-9f0e7c4a2d16
status: experimental
description: |
  Detects a long-lived outbound network session held by a root-owned process
  executing from a directory outside package manager control. Approved
  monitoring agents are excluded by the filter.
  Written following case MTB-IR-2026-0902.
references:
  - https://attack.mitre.org/techniques/T1071/001/
author: Meridian Trust Bank SOC
date: 2026-09-04
tags:
  - attack.t1071.001
  - attack.t1543.002
logsource:
  product: linux
  category: network_connection
detection:
  selection:
    User: 'root'
    Initiated: 'true'
    Image|startswith:
      - '/usr/local/bin/'
      - '/usr/local/sbin/'
  filter_approved:
    Image: '/opt/monagent/bin/monagent'
  condition: selection and not filter_approved
falsepositives:
  - Locally compiled administrative tooling installed outside the package
    manager. Should be inventoried; if it is not, that is its own finding.
level: high

```

A.3 CyberChef quick reference

Operations useful in this lab. Type the operation name into the search box at the top left of the Operations panel.

Operation	Category	Useful for
Extract IP addresses	Extractors	Pulling every address out of the egress log or a telemetry block
Sort	Utils	Ordering an extracted list
Unique	Utils	Collapsing a list of addresses down to the distinct ones
Count occurrences	Utils	Counting how many times one address appears across a log
Find / Replace	Utils	Reshaping a log block before pasting it into a spreadsheet
Regular expression	Regex	Extracting one field from every line, such as every dst_ip value
Defang IP Addresses	Networking	Making addresses safe before they go into a report
From Base64	Data format	Decoding an encoded payload string
SHA2	Hashing	Recomputing a file hash for verification

A.4 If something is not working

Problem	What to do
An ATT&CK sub-technique page returns not found	The identifier splits across the path. T1505.003 is at attack.mitre.org/techniques/T1505/003/ , not /T1505.003/ . This catches everyone once.
A technique name on the site does not match this document	MITRE revises names between releases. Use the name on the site today and note the difference. If a tactic says Defense Evasion anywhere, it now means Stealth.
attack.mitre.org is slow or will not load	Appendix A.1 prints every technique used in this incident with its official name and tactic. Say in your report that you used the printed reference.
My Sigma rule looks wrong and I cannot see why	Nine times out of ten it is indentation. Check for tab characters, and check that every level is indented by four spaces. A colon inside a value such as a file path must be inside quotes.
I cannot decide what logsource to use	There is no single correct answer, and that is fine. Pick the combination that best describes where the evidence came from, and state your assumption in the description field. Being explicit about the assumption is what is graded.
I cannot think of a false positive for my rule	That usually means the rule is too broad, or you have not asked who else does this legitimately. Backup software, configuration management and monitoring agents do things that look like attacker behaviour. Start there.
The hunt grid gives me a host I am not sure about	That is the intended experience for one of the five hosts. Section 6.4 holds the evidence that settles it. Read the asset record before you decide.
I cannot find the second large data transfer	Sort the egress log by the bytes_out column, largest first. The two rows you want are thousands of times larger than everything else and come from different hosts.

A.5 Working with an AI assistant

You may use an AI assistant on this lab, and you should. Use it the way a working analyst does.

- **Good use** - ask it to explain what a systemd unit does, what YAML indentation rules are, why a particular field exists in Sigma, or to review whether your executive summary reads clearly to a non-technical person.
- **Bad use** - asking it to write your four detection rules. You will get four rules that look plausible and you will not be able to defend a single design decision in them, which is the whole assessment.
- **Always verify** - AI assistants are frequently wrong about ATT&CK technique identifiers and about which tactic a technique belongs to. They are also often working from older ATT&CK releases, so they will tell you a technique sits under Defense Evasion. Check every identifier against attack.mitre.org or Appendix A.1 before it goes in your report.

Cite it. If an AI helped you reach a conclusion, note where and what you did to verify it. That is the standard a regulated bank holds analysts to, and it is the standard here.